



File Systems, Streams & Events in Node.js

Working with files, large data, and event-driven programming in Node.js — a practical deep-dive for backend developers ready to move beyond the basics.



File System



Streams



EventEmitter



Event Loop

Learning Objectives

What You'll Be Able to Do

By the end of this session, you'll have the practical skills to confidently handle files, process large data without crashing your server, and architect event-driven services in Node.js.

1 Read and write files

Use the built-in `fs` module for synchronous and asynchronous file operations.

2 Understand sync vs async

Know when each approach is appropriate and why it matters for server performance.

3 Work with Streams

Process large files efficiently using readable, writable, and piped streams.

4 Master EventEmitter

Understand and build event-driven applications using Node's core event system.

Why File Handling Matters in Backend Applications

Nearly every production Node.js application touches the file system. Understanding how to do it efficiently is not optional — it's foundational.

Config Files

Load environment settings at startup

Logging

Write application logs and audit trails

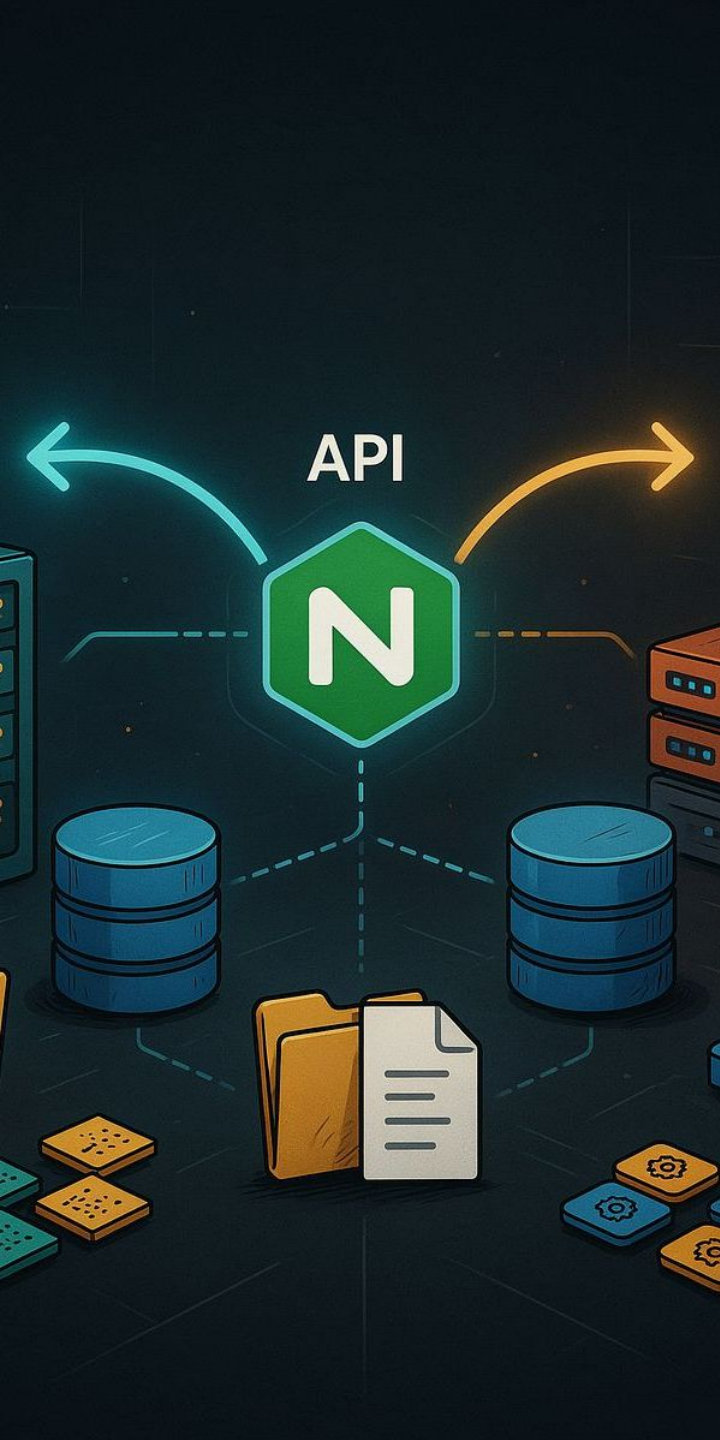
Uploads

Process user-uploaded files safely

Reports

Generate and export CSV/JSON data

 Key question: What happens if a Node.js server spends 10 seconds reading a huge file *synchronously*? The JavaScript thread is blocked — every other incoming request must wait. This is why async file I/O is not just a best practice, it's a requirement.



Chapter 1 — The fs Module

Node.js Built-in File System Module

Node.js ships with a powerful built-in `fs` module — no external packages required for core file operations. It offers both a legacy callback API and a modern Promise-based interface.

Legacy API

```
const fs = require('fs');
```

Uses callbacks. Still common in older codebases and low-level tooling.

readFile()

Read file contents

writeFile()

Create or replace a file

mkdir()

Create a directory

Modern Promise API

```
const fs = require('fs/promises');
```

Works seamlessly with `async/await`. Recommended for all new code.

appendFile()

Add to an existing file

readdir()

List directory contents

stat()

Get file metadata

Synchronous vs Asynchronous File Operations

Node.js offers both blocking and non-blocking APIs. Choosing the wrong one in a server context can stall your entire application for every concurrent user.

● Synchronous — Blocks the Thread

```
const data = fs.readFileSync(  
  'notes.txt',  
  'utf-8'  
);  
console.log(data);
```

Request Arrives

readFileSync Blocks

File Loads, Continue

Simple and readable, but the entire event loop pauses until the file is read. Only acceptable during startup initialization.

● Asynchronous — Non-Blocking

```
fs.readFile(  
  'notes.txt',  
  'utf-8',  
  (err, data) => {  
    console.log(data);  
  }  
);
```

Node kicks off the file read, then continues processing other requests. The callback fires when data is ready — the thread is never blocked.

- ⊗ Production rule: **Never use synchronous filesystem operations inside request-handling code.** Use them only at startup, before the server begins accepting connections.

Modern Pattern

fs/promises with async/await

The Promise-based `fs/promises` API is the recommended approach for modern Node.js. It eliminates deeply nested callbacks and integrates naturally with `async/await` and `try/catch` error handling.

Clean, Readable Code

```
const fs = require('fs/promises');

async function readNotes() {
  try {
    const data = await fs.readFile(
      'notes.txt',
      'utf-8'
    );
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

readNotes();
```

Why Prefer This Pattern?

→ Cleaner syntax

Reads top-to-bottom like synchronous code, without blocking.

→ Easier error handling

Standard `try/catch` covers both sync and async failures in one block.

→ No callback hell

Eliminates nested pyramid-of-doom structures common in older Node.js code.

Writing and Appending Files

Two key operations for writing data: `writeFile()` replaces the entire file each time, while `appendFile()` adds new content to the end. Choosing the right one depends on your use case.

`writeFile()` — Create or Replace

```
await fs.writeFile(  
  'students.txt',  
  '101,John\n'  
);
```

If the file exists, its content is **completely replaced**. If it doesn't exist, it's created. Ideal for generating fresh reports or overwriting config snapshots.



Generate Reports

Overwrite with fresh data on each run using `writeFile()`



Application Logs

Continuously grow log files using `appendFile()`



Audit Trails

Record every action without losing history with `appendFile()`

Working with Directories and File Metadata

Beyond reading and writing individual files, the `fs` module gives you full control over directory structures and file metadata — essential for building file management features.

Directory Operations

```
// Create directory (nested ok)
await fs.mkdir('reports', {
  recursive: true
});

// List contents
const files = await fs.readdir('reports');
console.log(files);
```

The `recursive: true` option creates all intermediate directories automatically — similar to `mkdir -p` in the shell.

File Metadata with `stat()`

```
const info = await fs.stat('notes.txt');
console.log(info.size);    // bytes
console.log(info.isFile()); // true/false
console.log(info.mtime);   // modified time
```

The `stat()` method returns a rich metadata object including file size, type, creation time, and last modification timestamp — useful for validation and monitoring.

- 📌 Mini challenge: Create a students directory, write a few student records to a file inside it, then use `readdir()` to display all files within the directory.

The Problem with Large Files

When you use `readFile()` on a small config file, it works perfectly. But what happens when that file is 5 GB?

❌ The Naive Approach

```
const data = await fs.readFile(  
  'bigfile.txt'  
);
```

Node.js attempts to load the **entire 5 GB file into memory at once**. This can crash your process, exhaust available RAM, and bring down other services running on the same machine.

✅ The Better Approach

Process the file in **small chunks** as it arrives from disk — never loading more than one chunk at a time into memory. Your application stays lean and responsive regardless of file size.



📌 This is exactly where **Streams** come in — the core Node.js abstraction for processing data incrementally.

Chapter 2 — Streams

What Are Streams?

A stream allows data to be processed **piece by piece** as it arrives, rather than waiting for the entire payload to be available. This is the key to handling large data efficiently in Node.js.

Traditional Approach

File → Entire contents loaded into Memory → Application processes

Works fine for small files. Fails or degrades badly for large ones.

Stream Approach

File → Chunk → App → Chunk → App → Chunk → App

Memory usage stays low and constant, regardless of total file size.

Think of `kubectl logs -f` or `tail -f`. You don't download the entire log file before reading it — log lines stream to your terminal as they're written.

Stream = process data as it arrives.

Four Types of Streams in Node.js

Node.js provides four fundamental stream types, each serving a distinct role in the data pipeline. Understanding these types lets you compose powerful, memory-efficient data flows.

1

Readable

Produces data that can be consumed. The source of the pipeline.

Example: `fs.createReadStream()`, HTTP request bodies

2

Writable

Consumes data written to it. The destination of the pipeline.

Example: `fs.createWriteStream()`, HTTP responses

3

Duplex

Both readable and writable simultaneously — two independent channels.

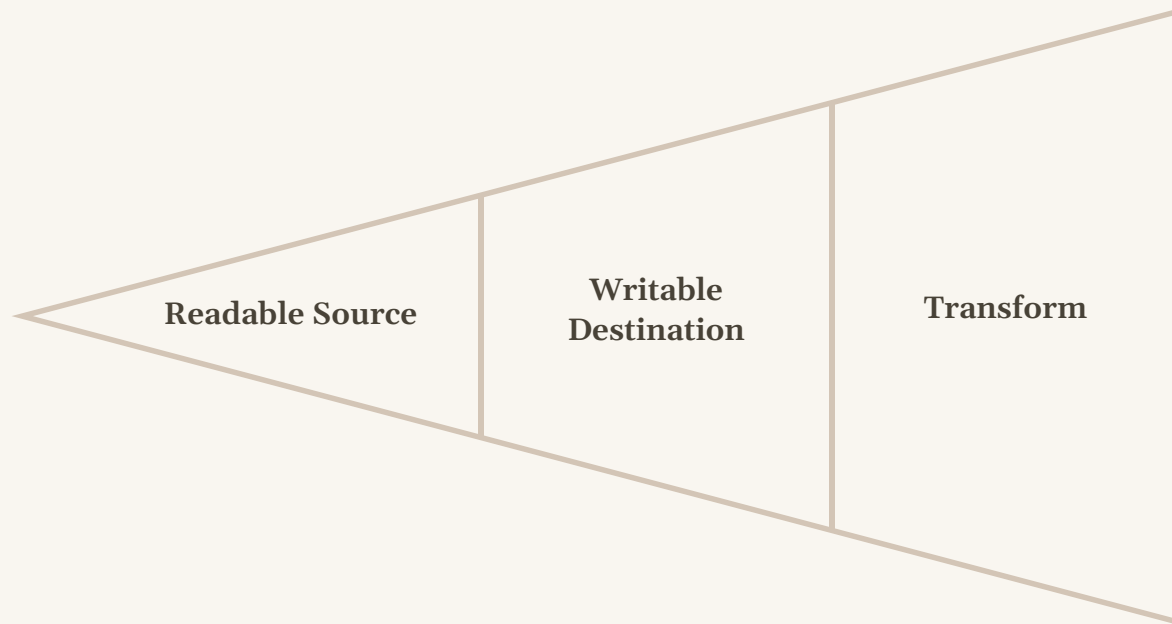
Example: TCP sockets, WebSocket connections

4

Transform

Reads input, modifies or processes it, and produces new output.

Example: `zlib.createGzip()` for on-the-fly compression



Readable and Writable Streams in Practice

Creating a file copy with streams illustrates the core pattern: open a readable stream on the source, open a writable stream on the destination, and forward chunks as they arrive.

Stream-Based File Copy

```
const fs = require('fs');

const readStream =
  fs.createReadStream('bigfile.txt');

const writeStream =
  fs.createWriteStream('copy.txt');

readStream.on('data', (chunk) => {
  console.log(
    `Received ${chunk.length} bytes`
  );
  writeStream.write(chunk);
});

readStream.on('end', () => {
  console.log('Read complete');
});
```

How the Flow Works



- ✓ The application never holds the entire file in memory — only one chunk at a time. A 5 GB file uses only a few kilobytes of RAM during the copy.

pipe(): The Elegant Way to Connect Streams

Manually wiring `data` events between streams works, but Node.js gives us a cleaner abstraction: `pipe()`. It connects a readable source directly to a writable destination in a single, expressive line.

✗ Manual Chunk Handling

```
readStream.on('data', chunk => {  
  writeStream.write(chunk);  
});  
readStream.on('end', () => {  
  writeStream.end();  
});
```

Verbose. You're responsible for managing flow control and teardown.

✓ With pipe()

```
fs.createReadStream('bigfile.txt')  
  .pipe(  
    fs.createWriteStream('copy.txt')  
  );
```

One line. Node handles chunk forwarding, flow control, and stream completion automatically.

Less Code

Removes boilerplate event wiring

Cleaner

Intent is immediately clear

Backpressure

Automatically slows the producer when the consumer is slower — prevents memory buildup

Streams Emit Events

Streams are not just data-processing pipelines — they are also **event emitters**. Every readable and writable stream fires events at key moments in its lifecycle, giving you precise control over how your application responds.

Listening to Stream Events

```
const stream =  
  fs.createReadStream('bigfile.txt');  
  
stream.on('data', chunk => {  
  console.log('Received chunk');  
});  
  
stream.on('end', () => {  
  console.log('Finished reading');  
});  
  
stream.on('error', err => {  
  console.error('Stream error:', err);  
});
```

Common Stream Events

data

Fires each time a new chunk of data is available for processing.

end

Fires when the entire source has been read — no more data is coming.

error

Fires if the stream encounters an error — always handle this to prevent crashes.

This event-based design is not coincidental — streams are built on top of **EventEmitter**, which is our next topic.

Chapter 3 — EventEmitter

The EventEmitter Pattern

Node.js uses the `EventEmitter` pattern as the foundation for streams, HTTP servers, process signals, and more. Understanding it unlocks the full power of Node's event-driven architecture.

1. Require

```
const EventEmitter =  
  require('events');
```

2. Create

```
const emitter =  
  new EventEmitter();
```

3. Subscribe

```
emitter.on('message',  
  () => console.log(  
    'Message received'  
  )  
);
```

4. Publish

```
emitter.emit('message');
```

EventEmitter: Publisher → Subscribers

The real power of `EventEmitter` is that a single emitted event can trigger **multiple independent listeners** simultaneously. This decouples your components — the publisher doesn't know or care who is listening.

How It Works

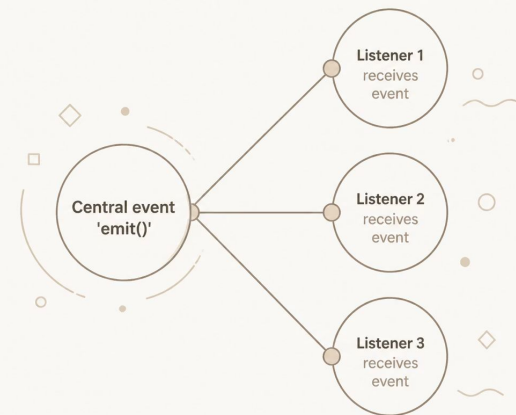
```
const emitter = new EventEmitter();

// Listener 1
emitter.on('message', (data) => {
  console.log('Listener 1:', data);
});

// Listener 2
emitter.on('message', (data) => {
  console.log('Listener 2:', data);
});

// Both fire
emitter.emit('message', 'Hello!');
```

Event Fan-Out Pattern



Building a Custom Event-Driven Service

Extending `EventEmitter` is how Node.js services are designed to be loosely coupled. Here's a realistic example: an order processing service where one event triggers multiple independent business actions.

Order Service Implementation

```
const EventEmitter = require('events');

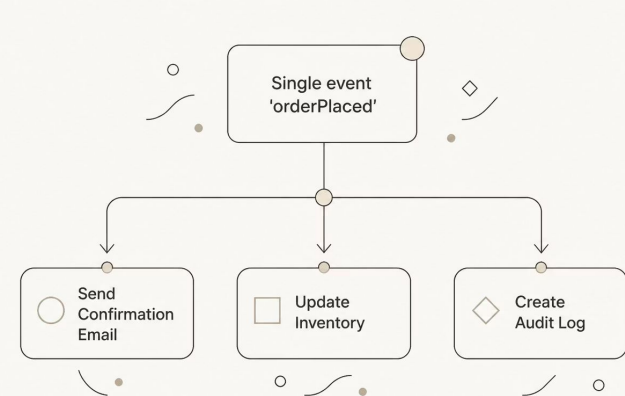
class OrderService extends EventEmitter {}
const orders = new OrderService();

orders.on('orderPlaced', order => {
  console.log(
    `Email sent for order #${order.id}`
  );
});

orders.on('orderPlaced', order => {
  console.log(
    `Inventory updated for #${order.id}`
  );
});

orders.emit('orderPlaced', {
  id: 101,
  item: 'Laptop'
});
```

One Event → Many Actions



This same pattern powers CI/CD webhooks, Kubernetes watch events, and message-driven microservices — just at a larger scale.

on() vs once() — Controlling Listener Lifetime

Not all listeners should run forever. Node.js provides two subscription methods with different lifecycle semantics — choosing the right one prevents memory leaks and unintended repeated executions.

on() — Persistent Listener

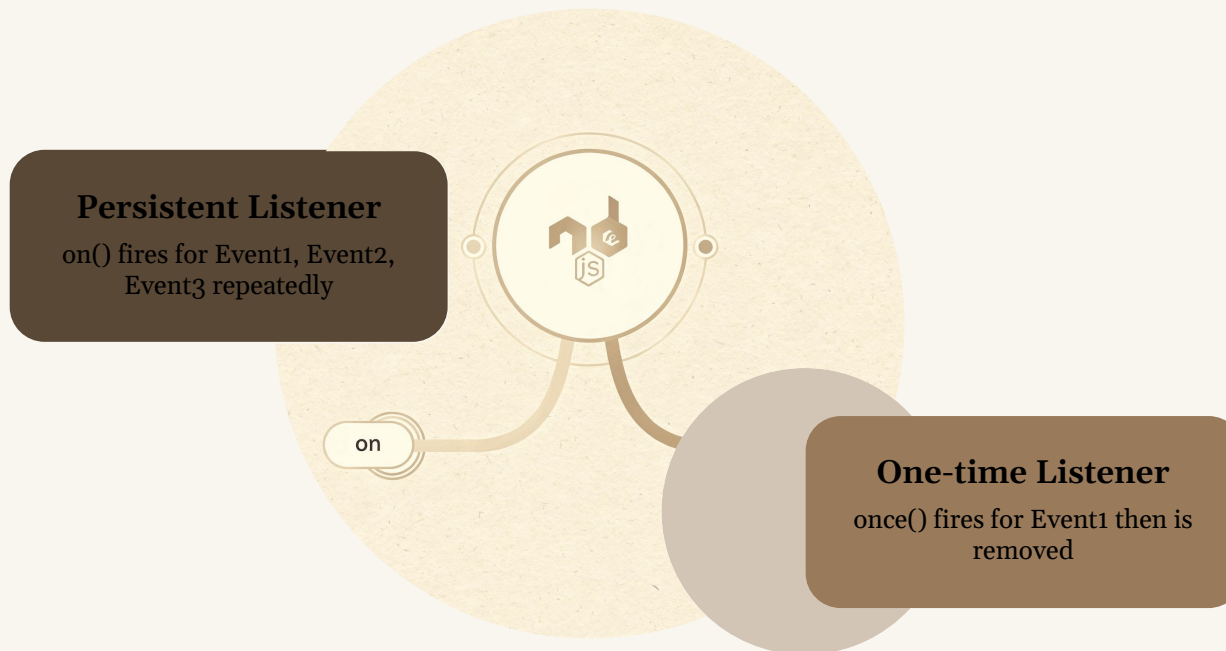
```
emitter.on('login', (user) => {  
  console.log(`${user} logged in`);  
});
```

Fires **every time** the event is emitted. Use for ongoing, repeatable events like incoming requests, new data chunks, or user actions that happen throughout the application's lifetime.

once() — One-Time Listener

```
emitter.once('startup', () => {  
  console.log("Server initialized");  
});
```

Fires **only on the first emission**, then automatically removes itself. Use for initialization events, one-time configuration loading, or any action that should never repeat.



Chapter Summary

Hour 2 — Key Takeaways

You've covered the three core systems that make Node.js a powerful backend platform for file-intensive and event-driven applications.



fs Module

Use `fs/promises` with `async/await` for clean, non-blocking file operations



Streams

Process large files in chunks using Readable, Writable streams and `pipe()` for backpressure



EventEmitter

Build decoupled, event-driven services where one action fans out to multiple independent handlers

- 📌 These three concepts are interdependent: Streams are built on EventEmitter, and efficient file handling requires understanding both. Together, they form the backbone of production-grade Node.js applications.

What's Next

Practice Challenges & Next Steps

The best way to solidify these concepts is to build with them. Try these hands-on exercises before the next session.

1 File Logger

Build a simple logger that appends timestamped entries to a `.log` file using `appendFile()`. Read back the last 10 lines on startup.

2 Large File Processor

Use `createReadStream()` and `pipe()` to copy a large file. Add a `Transform` stream that converts each line to uppercase as it passes through.

3 Event-Driven Order System

Extend the `OrderService` example. Add a `orderCancelled` event with listeners for refund processing, inventory restoration, and notification sending.

